

Real-Time EV Charging Load Forecasting and Dynamic Pricing Engine

Team Implementation Reference Guide

1. What This Document Is For

This document tells every team member exactly what they need to build and how the different parts of the system connect to each other. Before you write any code, please read through the sections relevant to your phase. The most important part is Section 2 — the shared data format. Everyone must follow it exactly, otherwise the pipeline will not work when we connect everything together.

The names assigned to each phase are temporary and subject to change after tonight's discussion. The task descriptions and interfaces are final.

2. How the System Works (Simple Overview)

Think of the system as an assembly line with 5 stations:

- Phase 1 reads the LOA charging dataset and sends it into Kafka, one message per charging record, simulating real-time data coming from charging stations.
- Phase 2 (Flink) picks up those messages, groups them into 15-minute time windows per station, and computes a summary called a feature vector. This is the input the model needs.
- Phase 3 takes each feature vector, runs it through the XGBoost model, and outputs two things: a predicted load and a dynamic price multiplier. The multiplier is applied to the real Ontario electricity price from IESO to get the final price signal.
- Phase 4 saves all results to PostgreSQL and watches for high-load situations. If multiple stations are overloaded at the same time, it sends a Grid Stress alert.

- Phase 5 wraps everything in Docker Compose so it all runs with one command, and shows a live dashboard with per-station load and prices.

The data flow is: LOA dataset → Kafka → Flink → feature vector → XGBoost/FastAPI → PostgreSQL + Dashboard.

3. The Feature Vector – The Most Important Interface

A feature vector is a summary of what happened at one charging station during one 15-minute window. It is the output of Phase 2 and the input of Phase 3. Everyone needs to know what it looks like.

Why 15 minutes?

The LOA dataset records data every 5 minutes. We group every three 5-minute records into one 15-minute window and compute statistics on them. This gives the model a richer summary than a single raw reading.

Example: what one feature vector looks like

The JSON below is one feature vector for station 1000604065, covering the 15-minute window starting at 2:00 PM on June 1, 2023.

```
{
  "station_id":          "1000604065",    // which station
  "time_bin":            "2023-06-01 14:00:00", // start of the 15-min
window
  "mean_kwh":            0.65,              // average electricity
delivered
  "variance_kwh":         0.12,              // how much it varied
  "rate_of_change":       0.03,              // load going up or down vs
last window
  "capacity_utilization_ratio": 0.72,        // how full the station is
(0.72 = 72%)
  "hour_of_day":          14,                // 2pm
}
```

```

    "day_of_week":          3,                // Thursday
    "anomaly_flag":        0,                // no bad sensor reading
detected
    "data_completeness":    0.95             // 95% of records had actual
charging
}

```

Field definitions

Field name	Type	Where it comes from	What it means
station_id	string	LOA raw data	The unique ID of the charging station
time_bin	datetime	Rounded from time_new	The start of the 15-minute window, e.g. 14:00, 14:15, 14:30
mean_kwh	float	Flink window average	Average electricity delivered in this 15-minute window
variance_kwh	float	Flink window calculation	How much the kwh values vary within the window. High variance = unstable load
rate_of_change	float	Flink computed	Difference between this window mean and the previous window mean. Positive = load is rising
capacity_utilization_ratio	float	Flink computed	mean_kwh divided by 15.0. Tells us how busy the station is. Above 1.0 means overloaded
hour_of_day	int	From time_new	Hour of day from 0 to 23. Helps the model learn time-of-day patterns
day_of_week	int	From time_new	0 = Monday, 6 = Sunday. Helps the model learn weekday vs weekend patterns
anomaly_flag	int	Flink anomaly detection	1 if any bad sensor reading was detected in this window, 0 if everything looks normal

data_completeness	float	Flink computed	Fraction of non-zero records in the window. 1.0 means all records had actual charging activity
-------------------	-------	----------------	--

Important note on capacity_utilization_ratio

The LOA dataset does not tell us the capacity of each station. We use a fixed default: 15.0 kWh per 15-minute window for all stations. So `capacity_utilization_ratio` = `mean_kwh` divided by 15.0. If the ratio goes above 1.0, the station is considered overloaded. This is a simplification that is fine for this prototype.

4. Kafka Topics — How the Phases Talk to Each Other

Kafka is the messaging system that connects all the phases. Think of each topic as a channel. One phase writes messages to a channel, and the next phase reads from it. These topic names are fixed — do not change them.

Topic name	Who writes to it	Who reads from it
ev-telemetry	Phase 1 (producer)	Phase 2 (Flink)
features	Phase 2 (Flink)	Phase 3 (FastAPI model)
dynamic-prices	Phase 3 (FastAPI model)	Phase 4 (PostgreSQL), Phase 5 (dashboard)
alerts	Phase 4 (alerting)	Phase 5 (dashboard)
data-quality	Phase 2 (Flink)	Phase 4 (PostgreSQL)

5. FastAPI Response Format

Phase 3 wraps the XGBoost model in a FastAPI web service. When Phase 2 sends a feature vector to it, it returns this JSON. Phase 4 and Phase 5 must expect exactly this format.

Example: the model received a feature vector for station 1000604065 and predicted it will deliver 0.81 kWh in the next window, which means the price should go up 24%.

```
{
  "station_id":      "1000604065",
  "time_bin":        "2023-06-01 14:00:00",
  "predicted_kwh":    0.81,           // how much electricity the model thinks
will be delivered
  "price_multiplier": 1.24,           // multiply the base IESO price by this
  "alert_level":      "normal"        // normal / warning / critical
}
```

Alert level rules: normal means CUR is below 0.7. warning means CUR is between 0.7 and 0.9. critical means CUR is above 0.9.

Price multiplier rules: below 0.7 CUR the multiplier is 1.0 (standard price). Between 0.7 and 0.9 it scales linearly up to 1.5. Above 0.9 it is fixed at 2.0 (double price to push users away).

6. PostgreSQL Tables

Phase 4 stores all results in PostgreSQL. These are the two tables. Phase 4 creates them. Phase 5 reads from them for the dashboard.

Table 1: inference_results

Stores every prediction the model makes, one row per station per 15-minute window.

```
CREATE TABLE inference_results (
  id                SERIAL PRIMARY KEY,
  station_id        VARCHAR(20),
  time_bin          TIMESTAMP,
  predicted_kwh      FLOAT,
  price_multiplier   FLOAT,
  alert_level        VARCHAR(10),
```

```
    created_at      TIMESTAMP DEFAULT NOW()
);
```

Table 2: grid_stress

Stores the regional stress score, computed every 15 minutes by aggregating all stations. If the overall stress score is above 0.8, a regional alert is triggered.

```
CREATE TABLE grid_stress (
  id          SERIAL PRIMARY KEY,
  time_bin    TIMESTAMP,
  stress_score FLOAT,
  station_count INT,
  triggered_alert BOOLEAN,
  created_at  TIMESTAMP DEFAULT NOW()
);
```

7. What Each Phase Needs to Build

Phase 1 — Kafka + Data Ingestion

Goal: get the LOA data flowing into Kafka so Phase 2 can consume it.

- Set up a single Kafka broker in Docker
- Create the ev-telemetry topic
- Write a Python producer that reads LOA.csv and sends each row as a JSON message to ev-telemetry
- Run one producer thread per station_id in parallel
- Messages must be published in time order per station

Each message looks like: { station_id, time_new, duration, kwh } — just the raw LOA fields, no processing needed here.

Phase 2 — Flink Feature Engineering

Goal: transform the raw Kafka stream into feature vectors and publish them to the features topic.

- Write a Flink job that subscribes to the ev-telemetry topic
- Group records into 15-minute tumbling windows per station_id
- Compute all fields in the feature vector schema from Section 3
- Use a 30-second watermark to handle late-arriving records
- If a record's kwh is more than 3 standard deviations from the station rolling mean, flag it as anomalous, replace it with the rolling mean, and publish it to the data-quality topic
- Publish completed feature vectors to the features topic

The output JSON must exactly match the feature vector format in Section 3. Field names are case-sensitive.

Phase 3 — XGBoost Model and FastAPI Service

Goal: train a model that predicts load and computes price multipliers, and serve it via a FastAPI endpoint.

- Prepare training data from LOA: aggregate raw records into 15-minute windows and compute all feature vector fields
- Split data chronologically: earlier portion for training, later portion for validation. No random shuffle.
- Train XGBoost with target variable: mean_kwh of the next 15-minute window (one window ahead)
- Implement the price multiplier and alert_level logic from Section 5
- Apply the multiplier to the IESO Ontario Energy Price to get the final per-kWh tariff
- Wrap everything in a FastAPI service that accepts a feature vector JSON and returns the response format from Section 5
- Also run the Peak Demand Alert experiment: test prediction accuracy at 5, 10, and 15-minute prediction horizons and report MAE at each horizon

The FastAPI service subscribes to the features topic, runs inference for each message, and publishes results to the dynamic-prices topic.

Phase 4 – PostgreSQL and Alerting

Goal: save all results to the database and implement the alerting logic.

- Set up PostgreSQL with TimescaleDB extension in Docker
- Create the two tables from Section 6
- Subscribe to the dynamic-prices topic and insert each message into inference_results
- Subscribe to the data-quality topic and log anomalous events
- Single-station alert: if alert_level is critical, publish an alert message to the alerts topic
- Grid Stress Indicator: every 15 minutes, compute the average capacity_utilization_ratio across all active stations. If the average exceeds 0.8, insert a row into grid_stress with triggered_alert = true and publish a regional alert to the alerts topic

Phase 5 – Docker Compose and Dashboard

Goal: wrap everything in Docker and build the live dashboard.

- Write a Docker Compose file that defines all five services: Kafka, Flink, FastAPI, PostgreSQL, Dashboard. All on the same Docker network.
- Dashboard must show: per-station load heatmap, current price multiplier per station, alert status (normal / warning / critical), Grid Stress Indicator score
- Dashboard subscribes to dynamic-prices and alerts Kafka topics and refreshes automatically
- Record a 2 to 3 minute demo video in Week 2 showing the full pipeline running
- Measure and report end-to-end latency: time from a message entering ev-telemetry to a price signal appearing in dynamic-prices

8. Week 1 Goals and Timeline

Phase	What it does	Week 1 goal	Can start now?
Phase 1 (Johnry)	Kafka + data producer	Kafka running in Docker. Producer replaying LOA data. ev-telemetry topic has messages flowing.	Yes – no dependency
Phase 2 (Deepta) deepta.ma...	Flink feature engineering	Flink job consuming ev-telemetry and outputting feature vectors to features topic. Share a sample JSON output.	Write logic now, connect to Kafka once Phase 1 is ready
Phase 3 (Helen) fangyu.lin...	XGBoost model + FastAPI	XGBoost trained on LOA data. FastAPI running locally, accepting a feature vector and returning predicted_kwh and price_multiplier.	Offline training can start now
Phase 4 (Zahid) khzahidha...	PostgreSQL + alerting	PostgreSQL running with tables created. Grid Stress Indicator logic written. Share DB schema screenshot.	DB design can start now
Phase 5 (Camilo) camilo.bell...	Docker Compose + dashboard	Docker Compose file with all 5 services defined. Dashboard UI skeleton running. Share a screenshot.	UI and Docker skeleton can start now

Week 2 (June 8 to 14): full integration. All phases connected end to end. Demo recording and latency benchmarking.

Final deadline: June 22.

9. Rules Everyone Must Follow

- Do not change any field name in the feature vector or any Kafka topic name without telling the group first. One change breaks everyone downstream.
- If you are unsure about any interface, ask in the group chat before writing code.

- At the end of Week 1, share your deliverable in the group chat so everyone knows the status.